

“Case File Management & Analytics Portal”

A Project Report Submitted in the partial fulfillment of the
requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

**DEPARTMENT OF COMPUTER SCIENCE AND
INFORMATION TECHNOLOGY**

By

P.Ruhitha 2520080053

M.Ananya 2520080069

S.Himasree 2520090181

Under the Esteemed Guidance of

D.Divya Priya

Assistant Professor

**Department of Computer Science and Information
Technology**



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Declaration

The Project Report entitled **“Case File Management & Analytics Portal”** is a record of bonafide work of **P.Ruhitha – 2520080053, M.Ananya – 2520080069, S.Himasree - 2520090181** submitted in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 11, during 2026–2027, Trimester 4, at KLH University. The results embodied in this report have not been copied from any other student, department, or institution.

P.Ruhitha – 2520080053

M.Ananya – 2520080069

S.Himasree - 2520090181

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

CERTIFICATE

This is to certify that the project report entitled **“Case File Management & Analytics Portal”** is a bonafide work done and submitted by **P.Ruhitha – 2520080053, M.Ananya – 2520080069, S.Himasree -2520090181** in partial fulfillment of the requirements of the course DATABASE SYSTEMS ENGINEERING AND DISTRIBUTED BACKEND DEVELOPMENT (25CS1302E), Section 10, Team 11, at KLH University during 2026–2027, Trimester 4.

Signature of the Guide
D. Divya Priya

Signature of the Course Coordinator

Signature of the HOD
Dr. Kaja Shareef (CSIT)

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who assisted us in one way or the other in completing this project.

Our greatest appreciation and sincere thanks to our guide **D. Divya Priya**, Department of Computer Science and Information Technology, for her tremendous support, encouragement, valuable guidance and continuous assistance throughout the development of the project.

We express our gratitude to **Dr. Kaja Shareef**, Head of the Department of Computer Science and Information Technology, for providing us with adequate facilities, ways and means by which we were able to complete this project-based lab.

We thank all the members of the teaching and non-teaching staff, and everyone who assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

P.Ruhitha **2520080053**

M.Ananya **2520080069**

S.Himasree **2520090181**

ABSTRACT

The Case File Management & Analytics Portal is a web-based application designed to help law firms, lawyers and their clients manage legal cases, court hearings, case documents and judgments in a single, database-backed platform. In many practices, case information is still scattered across paper files, spreadsheets and messaging apps, which makes it difficult to see which cases are active, when the next hearing is scheduled, which documents belong to which case, or how a case was finally decided.

The proposed system provides a role-based web interface. Lawyers can register and track cases, schedule hearings before a judge, record case documents and judgments, and view an analytics dashboard summarising their workload. Clients can log in to follow the cases filed on their behalf, their hearings, documents and outcomes. Every record is stored in a MySQL relational database, and the relationships between users, clients, lawyers, judges, cases, hearings, documents and judgments are enforced through primary keys, foreign keys and UNIQUE constraints.

The application follows a distributed, service-oriented design. A React (Vite) single-page application is the user interface. A FastAPI backend handles authentication, session management and analytics. Three independent Spring Boot microservices own the business domain: a User Service (users, clients, lawyers), a Case Service (cases) and a Record Service (hearings, documents, judgments). The services communicate with each other only through REST calls — the Case Service validates clients and lawyers by calling the User Service, and the Record Service validates cases by calling the Case Service — and never read each other's data directly.

The system was built and verified in incremental phases — database design, the three microservices, service-to-service communication, frontend integration and live analytics — with each phase tested against the running MySQL database. The result is a centralised platform that improves visibility of case progress, reduces missed or duplicated information, and demonstrates the practical use of database design together with distributed backend development.

INDEX

S.No.	Chapters	Topics	Page No.
	Acknowledgement		4
	Abstract		5
1	Introduction	1.1 Background of the Project 1.2 Problem Statement 1.3 Scope of the Project 1.4 Objectives 1.5 Importance of the Application 1.6 Target Users / Audience	8
2	System Requirements	2.1 Hardware Requirements 2.2 Software Requirements 2.3 Development Tools and Frameworks	10
3	Technology Stack	3.1 Frontend 3.2 Backend 3.3 Database 3.4 Version Control 3.5 Authentication and Security Libraries	11
4	System Architecture	4.1 High-Level Architecture Diagram 4.2 Description of Each Layer 4.3 Deployment Architecture	12
5	Design	5.1 Database Design Principles 5.2 ER Diagram / Database Schema 5.3 Design Decisions and Assumptions	14
6	Implementation	6.1 Module-wise Implementation 6.2 Front-end Logic 6.3 API Integration 6.4 Authentication and Authorization	16
7	Features	7.1 List of Main Features 7.2 How Each Feature Works	18
8	Testing	8.1 API Testing 8.2 Integration Testing 8.3 Tools Used for Testing 8.4 Test Cases and Results	19
9	Deployment	9.1 Steps to Deploy 9.2 Environment Configuration 9.3 Hosting of Frontend and Backend	21
10	Challenges & Limitations	10.1 Issues Faced During Development 10.2 Solutions Applied 10.3 Current Limitations or Known Bugs	22
11	Future Enhancements	11.1 Planned Features 11.2 Possible Integrations or Optimizations	23
12	Conclusion	12.1 Summary of the Project 12.2 What Was Achieved 12.3 Skills Learned	24

S.No.	Chapters	Topics	Page No.
13	References	Documentation, tools, and resources used	25
14	Appendices	Sample Code Snippets Installation / Setup Instructions Default Login Credentials API Reference Summary Screenshots	26

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Legal practice generates a large amount of information that must be tracked carefully: who the client is, which lawyer is handling the matter, what the case is about, when the next hearing takes place, which judge is presiding, what documents have been filed, and how the court finally decided. In many small and medium practices this information is still kept in paper files, personal notebooks, spreadsheets and chat messages. That works while the number of cases is small, but it breaks down quickly: a hearing date is missed, a document cannot be found, or a client keeps calling the office to ask for a status update.

The Legal Case Management System was built to replace this informal coordination with a single, database-backed source of truth. Every user, client, lawyer, judge, case, hearing, document and judgment is stored in one relational database, and the system itself — rather than a person cross-checking files — is responsible for keeping those records consistent, for example by refusing to create a case for a client or lawyer that does not exist.

1.2 Problem Statement

Given a legal practice that handles many cases for many clients across several courts, the system must reliably answer the following questions: which cases are currently active, pending or closed; which hearings are scheduled and before which judge; which documents and judgments belong to a particular case; and what the overall workload looks like. Answering these manually does not scale, is error-prone, and gives clients no visibility into the progress of their own matters.

1.3 Scope of the Project

The system covers the case lifecycle from registration to judgment: managing users (clients and lawyers) with role-based access; filing and maintaining cases with a case number, type, status, filing date and court; scheduling hearings against a judge, court room and date; recording case documents and their metadata; recording judgments with a decision, summary and remarks; and presenting an analytics dashboard that always reflects the current database. Payment and billing, e-filing with real courts, multi-firm tenancy and a native mobile application are out of scope for this submission.

1.4 Objectives

- Maintain one consistent database of clients, lawyers, judges, cases, hearings, documents and judgments.
- Provide role-based access so that lawyers and clients see the functionality appropriate to them.
- Split the business domain into independent microservices (User, Case, Record) that cooperate only through REST.
- Validate relationships at service level — a case needs a valid client and lawyer; a hearing, document or judgment needs a valid case.

- Provide an analytics dashboard (cases by status, type and month; hearings; documents; judgments) that is always computed from the current database state.
- Demonstrate database design, constraints and distributed backend development in one working application.

1.5 Importance of the Application

By centralising case data, the system removes guesswork from day-to-day practice management. Lawyers gain a live view of their caseload and upcoming hearings, and managers can see at a glance how many cases are open, pending or closed. Clients gain transparency into their own matters without having to contact the office for every update. For the course, the project also shows how a monolithic application can be decomposed into services that own their own data and communicate over well-defined APIs.

1.6 Target Users / Audience

The system serves two roles. **Lawyers** use it to manage cases, hearings, documents and judgments and to monitor analytics. **Clients** use it to follow the cases filed on their behalf, including hearings, documents and judgments associated with those cases.

CHAPTER 2: SYSTEM REQUIREMENTS

2.1 Hardware Requirements

Because the system runs five lightweight processes (React dev server, FastAPI, and three Spring Boot services) plus a MySQL server, the hardware needs are modest but slightly higher than a single-process application. A development or demonstration machine with a dual-core processor, 8 GB of RAM (4 GB is the practical minimum, as each JVM service uses a few hundred MB) and about 1 GB of free disk space for dependencies, build outputs and database files is sufficient to run the whole stack on one computer.

2.2 Software Requirements

- Windows operating system with PowerShell (the startup guide in this project is written for Windows PowerShell)
- MySQL Server 8.0 or later, running as a service (MySQL Workbench is only a client and does not start the server)
- Java JDK 17 or later (required by Spring Boot 3.x) to run the three microservice JAR files
- Apache Maven 3.9.6 (bundled inside the project folder) to rebuild the microservices when code changes
- Python 3 with a virtual environment (venv) for the FastAPI backend
- Node.js and npm for the React/Vite frontend
- A modern web browser (Chrome, Edge or Firefox)

2.3 Development Tools and Frameworks

React.js (bootstrapped with Vite) was used for the frontend; FastAPI with Uvicorn for the authentication and analytics backend; Spring Boot with Maven for the User, Case and Record microservices; MySQL as the relational database; Postman/Swagger UI and PowerShell's Invoke-RestMethod for API testing; and Visual Studio Code with MySQL Workbench as the development environment for editing code and inspecting the database directly.

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend

The frontend is a single-page application built with React.js and bundled with Vite. It runs on port 5173. Shared application state — the logged-in user, the list of cases, hearings, documents, judgments and the analytics data — is kept in a central React Context (AppContext). Two API helper modules (api.js for the FastAPI backend and microservicesApi.js for the Spring Boot services) keep every HTTP call in one place so that pages never hard-code URLs. Charts on the Analytics page are drawn with Recharts, and styling is hand-written CSS using shared design tokens.

3.2 Backend

The backend is split into two kinds of components:

- **FastAPI (Python, port 8000)** — handles registration, login, session handling and the /analytics/overview endpoint, which queries the database on every request.
- **Spring Boot microservices (Java, Maven)** — the User Service (port 8081) manages users, clients and lawyers; the Case Service (port 8082) manages cases; the Record Service (port 8083) manages hearings, documents and judgments. Each service is organised into controller, service, repository/entity and DTO layers, and is packaged as an executable JAR.

3.3 Database

MySQL 8 stores all persistent data in the database legal_case_management across eight related tables: users, clients, lawyers, judges, cases, hearings, documents and judgments. Referential integrity is enforced with FOREIGN KEY constraints, and uniqueness with UNIQUE constraints (one e-mail per user, one case number per case, one bar registration number per lawyer, and one judgment per case).

3.4 Version Control

The whole project is kept under a single project root containing the backend, case-file (frontend) and microservices folders, so that it can be tracked as one repository with Git. <https://github.com/Himasree24/Case-file-management-Analytics-portal.git>

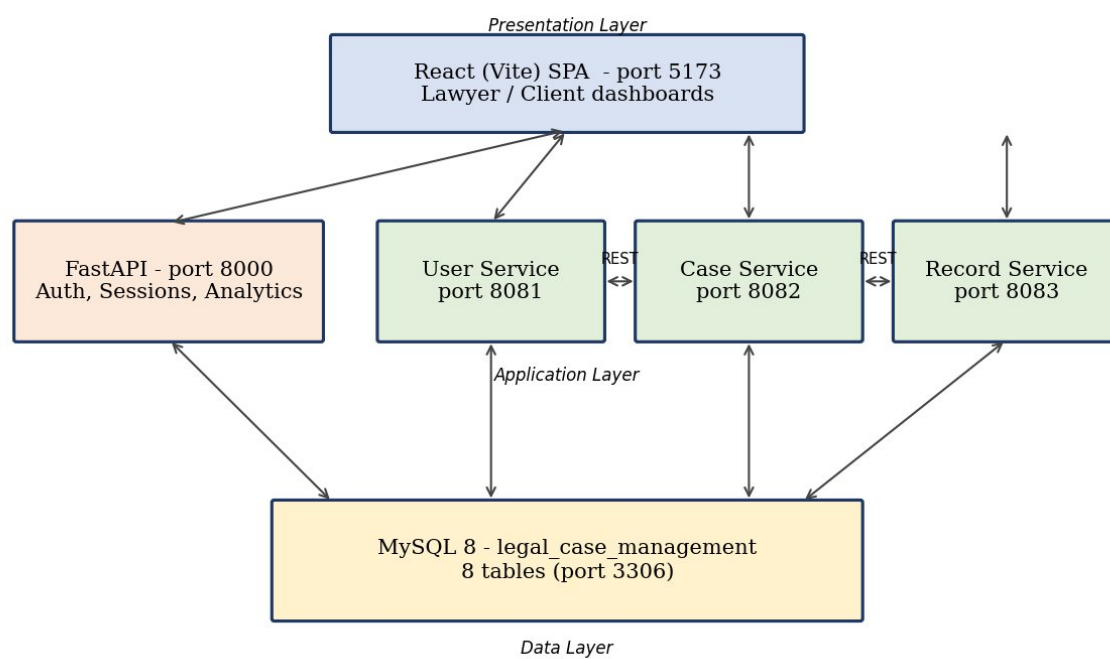
3.5 Authentication and Security Libraries

Passwords are never stored in plain text; they are hashed with the Argon2id algorithm and only the hash is saved in users.password_hash. After a successful login, the FastAPI backend creates a server-side session and returns a session token, which the React application sends in the X-Session-Token header on every protected request. Role information (lawyer or client) stored with the user is used by the frontend and backend to restrict what each role can see and do.

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The system follows a layered, service-oriented architecture that separates the user interface, the business logic and the persisted data. Figure 4.1 shows the layers and how requests flow between them.



4.2 Description of Each Layer

Presentation Layer. The React single-page application never talks to MySQL directly. Every piece of data it shows comes from calling REST APIs over HTTP — FastAPI for authentication and analytics, and the Spring Boot services for clients, lawyers, cases, hearings, documents and judgments.

Application Layer. FastAPI provides login, sessions and analytics. The three Spring Boot microservices implement the domain logic. Service-to-service communication uses REST: when a case is created or updated, the Case Service calls the User Service to confirm that the client and the lawyer exist; when a hearing, document or judgment is created or updated, the Record Service calls the Case Service to confirm that the case exists. Aggregated endpoints such as `/cases/{id}/with-users` and `/records/hearings/{id}/with-case` combine data from two services into one response. No service shares Java classes or Maven dependencies with another, so each can be built and deployed independently.

Data Layer. The MySQL database `legal_case_management` stores all persistent data. The services do not read each other's data directly; cross-service information is always obtained through the owning service's API.

4.3 Deployment Architecture

For development and demonstration, every component runs as a separate local process on one machine: React/Vite on port 5173, FastAPI on 8000, User Service on 8081, Case Service on 8082, Record Service on 8083, and MySQL on its default port 3306. The recommended start order is MySQL → FastAPI → User Service → Case Service → Record Service → React, because the Case Service depends on the User Service and the Record Service depends on the Case Service. No containerisation or CI/CD pipeline was configured for this submission.

CHAPTER 5: DESIGN

5.1 Database Design Principles

The schema was normalised so that each fact is stored in exactly one place and every relationship between entities is expressed through a foreign key rather than by duplicating data. Integrity is enforced in the database itself — using PRIMARY KEY, FOREIGN KEY and UNIQUE constraints — so that invalid data such as a duplicate e-mail, a duplicate case number, or a hearing that points to a non-existent case is rejected at the database level even if a bug in application code tried to insert it.

5.2 ER Diagram / Database Schema

The database consists of eight related tables, shown in Figure 5.1.

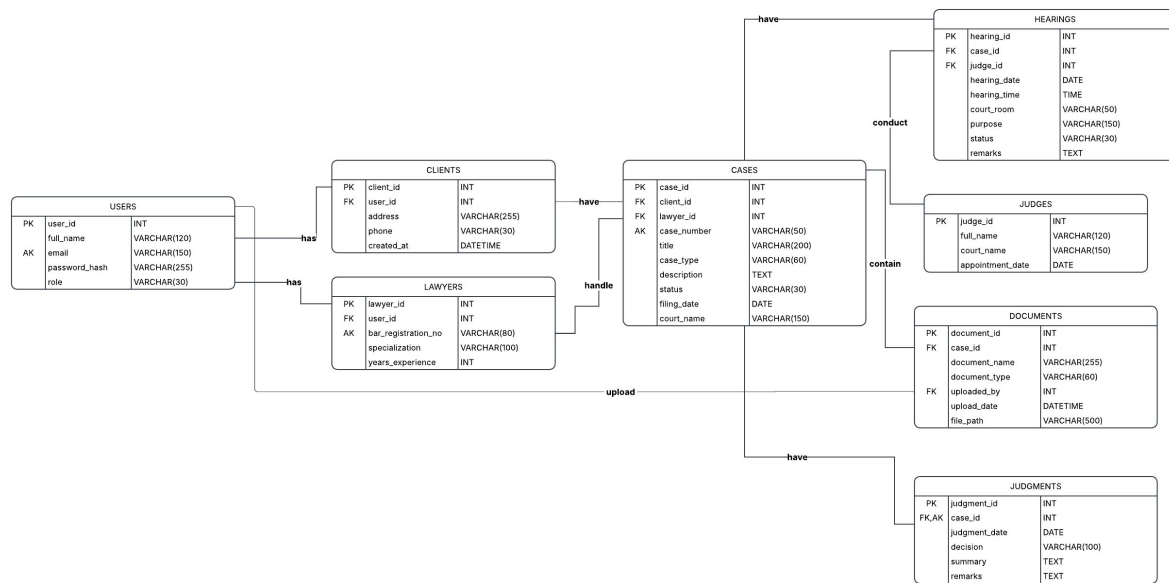


Figure 5.1: Entity-Relationship diagram of the eight-table database schema

users stores every account with a hashed password and a role (client or lawyer). **clients** and **lawyers** are profile tables, each linked to exactly one user through `user_id`; a client stores an address and phone number, while a lawyer stores a unique bar registration number, specialisation and years of experience. **judges** holds the judges' name, court and appointment date. **cases** links one client and one lawyer to a case with a unique case number, title, type, description, status, filing date and court. **hearings** records a scheduled hearing of a case before a judge, with date, time, court room, purpose, status and remarks. **documents** stores document metadata (name, type, uploader, upload date and file path) for a case. **judgments** stores the final decision, summary and remarks for a case; its `case_id` is UNIQUE, so a case can have at most one judgment.

Table	Key constraints	Relationships
users	PK user_id; UNIQUE email	1 : 1 clients / lawyers; 1 : N documents (uploaded_by)
clients	PK client_id; FK user_id	1 : N cases

Table	Key constraints	Relationships
lawyers	PK lawyer_id; FK user_id; UNIQUE bar_registration_no	1 : N cases
judges	PK judge_id	1 : N hearings
cases	PK case_id; UNIQUE case_number; FK client_id, lawyer_id	1 : N hearings, documents; 1 : 1 judgment
hearings	PK hearing_id; FK case_id, judge_id	N : 1 cases; N : 1 judges
documents	PK document_id; FK case_id, uploaded_by	N : 1 cases; N : 1 users
judgments	PK judgment_id; FK case_id (UNIQUE)	1 : 1 cases

5.3 Design Decisions and Assumptions

- **Microservices own their data domains.** User, client and lawyer data belong to the User Service; case data to the Case Service; hearing, document and judgment data to the Record Service. Other services access that data only through REST, never directly.
- **Validation through REST.** Foreign-key correctness is checked at service level (HTTP 400 Bad Request if the referenced client, lawyer or case does not exist), in addition to the database FOREIGN KEY constraints.
- **Documents are stored as metadata.** The documents table stores a file_path and descriptive fields; the system does not include a physical file upload/download service. The View action therefore displays the stored information and clearly reports when no actual file is available.
- **The database is the source of truth for analytics.** Analytics values are computed from the database on every request and are never cached or hard-coded in React.

CHAPTER 6: IMPLEMENTATION

6.1 Module-wise Implementation

The backend is organized into four modules:

- **Authentication & Analytics module (FastAPI).** Registers users, verifies Argon2id password hashes, creates server-side sessions, and exposes GET /analytics/overview, which counts cases by status, type and month and summarizes hearings, documents and judgments directly from MySQL.
- **User Service (port 8081).** CRUD and listing for users, clients and lawyers (/users, /clients, /lawyers), plus a health endpoint (/users/health).
- **Case Service (port 8082).** CRUD for cases (/cases). Before creating or updating a case it calls the User Service through a UserServiceClient (Spring RestClient) to check that the client and lawyer exist. GET /cases/{id}/with-users returns the case together with its client and lawyer details.
- **Record Service (port 8083).** CRUD for hearings, documents and judgments (/records/hearings, /records/documents, /records/judgments). It uses a CaseServiceClient to check the case exists before saving, and exposes aggregated endpoints such as /records/hearings/{id}/with-case.

Each Spring Boot service follows the same pattern: a controller receives the request, a service class applies the validation and business rules, a repository reads or writes the database, and DTO classes define the JSON that is returned to the caller.

6.2 Front-end Logic

The frontend uses a React Context (AppContext) as the single shared state container. It holds the cases, hearings, documents, judgments, clients and lawyers lists as well as analyticsData, analyticsLoading and analyticsError. Add, update and delete actions (for example addCase, updateCase and deleteCase) call the matching microservice, update the local list, and then call loadAnalytics() so that the analytics page is refreshed from the database instead of being calculated locally. A formatting function (formatCase) accepts both camelCase and snake_case field names so the UI works with the field names actually returned by the services. Role-based routing separates the lawyer and client dashboards.

6.3 API Integration

All HTTP calls are made through two helper modules. api.js talks to FastAPI and automatically attaches the X-Session-Token header; microservicesApi.js talks to the User, Case and Record services on ports 8081, 8082 and 8083. Pages call these helpers through the context functions, so no page hard-codes a URL or duplicates fetch logic. If a request fails — for example because a service is not running — the application shows a meaningful error rather than silently showing zeros.

6.4 Authentication and Authorization

A user registers with a name, e-mail, password and role. The password is hashed with Argon2id before it is stored. On login the backend verifies the hash and creates a session; the session

token is sent in the X-Session-Token header on protected requests such as analytics. The role (lawyer or client) determines which dashboard and which operations are available: lawyers manage cases, hearings, documents and judgments, while clients are restricted to viewing information associated with their own cases. Because sessions are kept in the memory of the FastAPI process, restarting FastAPI requires users to log in again; the MySQL data is unaffected.

CHAPTER 7: FEATURES

7.1 List of Main Features

Lawyer-facing features:

- Dashboard with the lawyer's cases and upcoming hearings
- Create, view, edit and delete cases (case number, title, type, description, status, filing date, court)
- Schedule, edit and delete hearings, with a judge, date, time and court room
- Record and manage case documents and judgments
- View clients and lawyers lists and details
- Analytics dashboard that updates live when cases or related records change

Client-facing features:

- Client dashboard listing the client's own cases
- View hearings for those cases
- View documents and use the document View action
- View judgments recorded for those cases

7.2 How Each Feature Works

Case management. When a lawyer saves a new case, the React app sends it to the Case Service. The Case Service calls the User Service to confirm that the client and lawyer exist, then saves the case in MySQL. If either does not exist, the request is rejected with HTTP 400. Case numbers are UNIQUE, so a duplicate case number is rejected by the database.

Hearings, documents and judgments. These are handled by the Record Service. Before saving, it calls the Case Service to verify that the referenced case exists. Aggregated endpoints (for example `/records/judgments/{id}/with-case`) return the record together with its case so the UI can show both without extra calls.

Live analytics. The Analytics page shows total cases, cases by status (Active, Pending, Under Review, Closed, etc.), cases by type, cases by month, hearings, documents and judgments. These numbers come from `GET /analytics/overview`, which runs SQL against the current database each time. After a case is created, updated or deleted, the frontend calls `loadAnalytics()` again. For example, with 20 cases in the database, adding one Active case changes the total to 21 and increases the Active count by one; changing it to Closed decreases Active and increases Closed while the total stays 21; deleting it returns the total to 20.

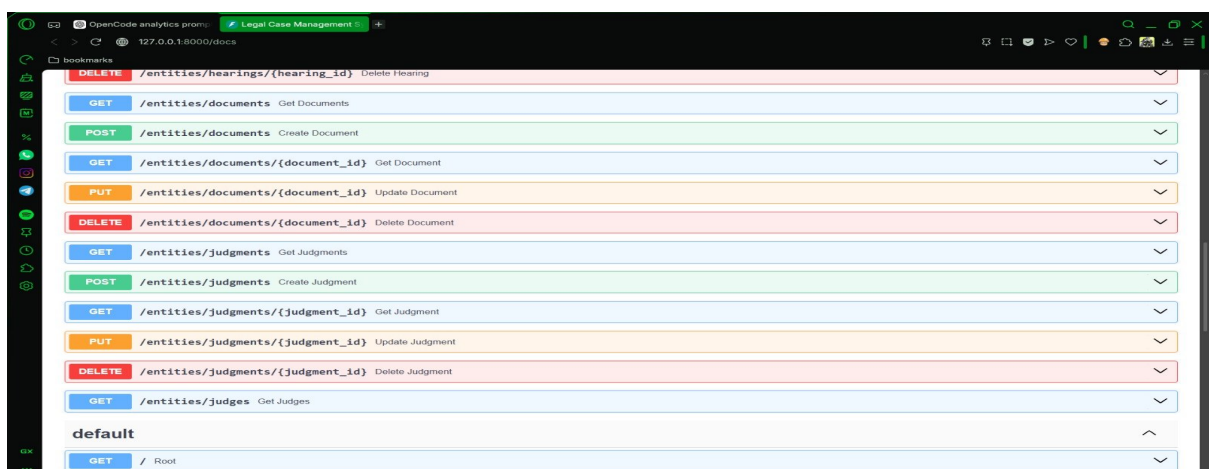
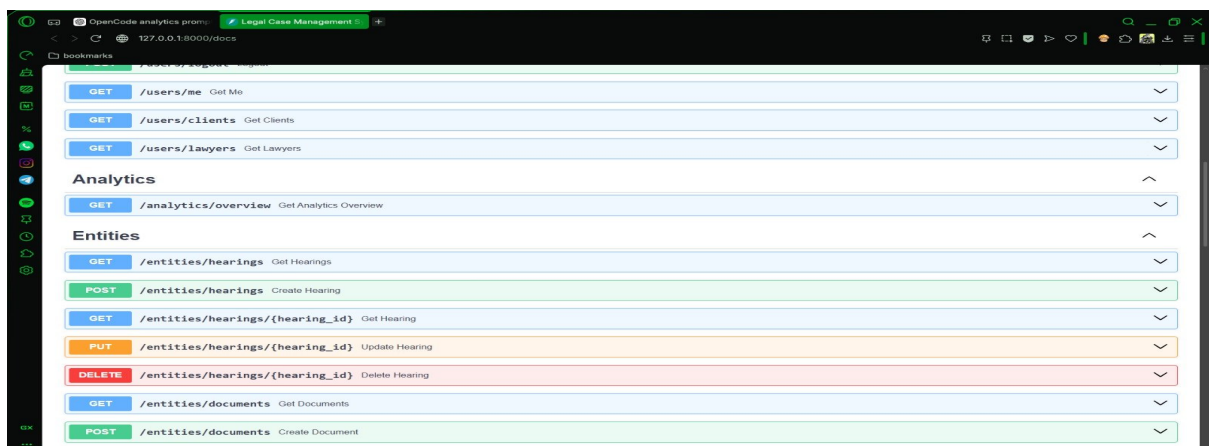
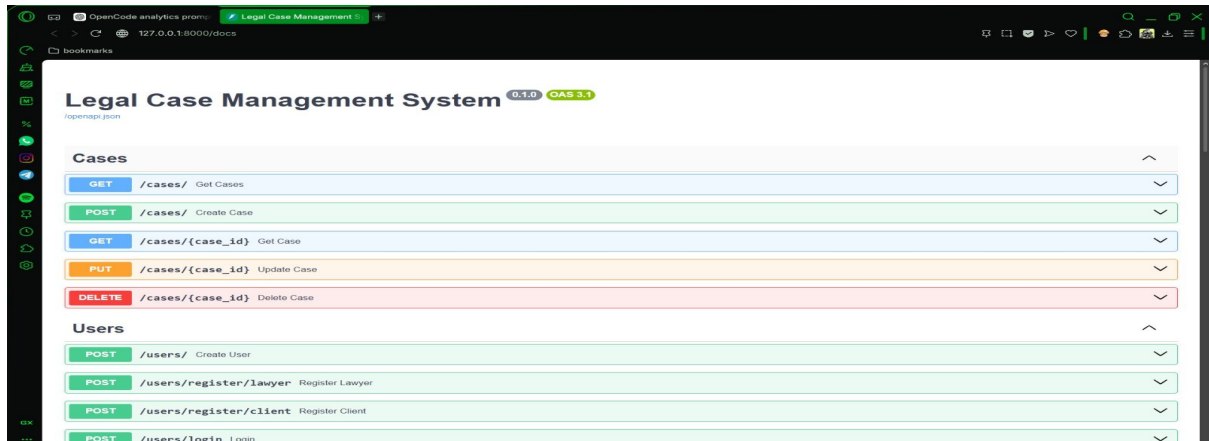
Document viewing. Because the system stores document metadata and a file path rather than the file itself, clicking View shows the stored document details and, where an accessible file location exists, opens it; otherwise a clear message states that the file is not available instead of failing silently.

Role-based access. Lawyers and clients are shown different dashboards and menus. Clients can view information related to their own cases but do not get the lawyer-only create, edit and delete operations.

CHAPTER 8: TESTING

8.1 API Testing

Each microservice exposes a health endpoint and CRUD/listing endpoints that were exercised directly, independently of the React frontend. Requests were issued with PowerShell's Invoke-RestMethod, the Swagger UI provided by FastAPI (<http://127.0.0.1:8000/docs>) and a browser, for example:



8.2 Integration Testing

Beyond isolated API calls, the system was tested end to end against the real, running MySQL database at every stage: first each microservice on its own, then the service-to-service calls (Case → User and Record → Case), then the React frontend against all services together. The existing database was never reset during these tests; the verified record counts were cases 20, users 17, clients 7, lawyers 9, hearings 7, documents 5 and judgments 4. The frontend was also compiled successfully with `npm run build` (2441 modules transformed).

8.3 Tools Used for Testing

- PowerShell Invoke-RestMethod — scripted API checks and health checks
- Swagger UI (FastAPI /docs) and Postman — manual endpoint testing
- MySQL Workbench / SQL queries — direct verification of table contents and constraints
- Manual browser testing — verifying the React frontend’s behaviour with lawyer and client logins

8.4 Test Cases and Results

A representative sample of the test cases is summarised below.

#	Test Case	Expected Result	Actual Result
1	GET /users/health on User Service	service = user-service, status = UP	Pass
2	GET /cases/health on Case Service	service = case-service, status = UP	Pass
3	GET /records/health on Record Service	service = record-service, status = UP	Pass
4	GET / on FastAPI	{"message": "Legal Case Management API is working!"}	Pass
5	GET /cases	All 20 cases returned	Pass
6	GET /clients and /lawyers	7 clients and 9 lawyers returned	Pass
7	GET hearings, documents, judgments	7 hearings, 5 documents, 4 judgments returned	Pass
8	GET /cases/1/with-users	Case combined with client and lawyer details from User Service	Pass
9	GET /records/hearings/6/with-case	Hearing combined with case details from Case Service	Pass
10	GET /records/documents/1/with-case and /records/judgments/1/with-case	Record combined with case details	Pass
11	Insert a case with an existing case_number	Rejected by the UNIQUE constraint	Pass
12	Frontend production build (npm run build)	Build completes without errors	Pass
13	Analytics lifecycle: add case → change status → delete case	Total 20 → 21; Active/Closed change; back to 20	[Record result after demo]
14	Client Documents → View	Opens document or shows a clear “file not available” message	[Record result after demo]

CHAPTER 9: DEPLOYMENT

9.1 Steps to Deploy

The application is deployed locally for development and demonstration. Do not run Project.sql again when restarting; the database already exists and holds the data. Each command below is run in its own terminal, which must stay open:

- Start the MySQL server (verify it is running; Workbench alone does not start it).
- Terminal 1 – FastAPI: cd backend, then `.\venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000`
- Terminal 2 – User Service: cd microservices\user-service, then `java -jar target\user-service-1.0.0.jar` (port 8081)
- Terminal 3 – Case Service: cd microservices\case-service, then `java -jar target\case-service-1.0.0.jar` (port 8082)
- Terminal 4 – Record Service: cd microservices\record-service, then `java -jar target\record-service-1.0.0.jar` (port 8083)
- Terminal 5 – React: cd case-file, then `npm run dev` (port 5173)
- Run the health checks, open `http://localhost:5173` in a browser and log in.

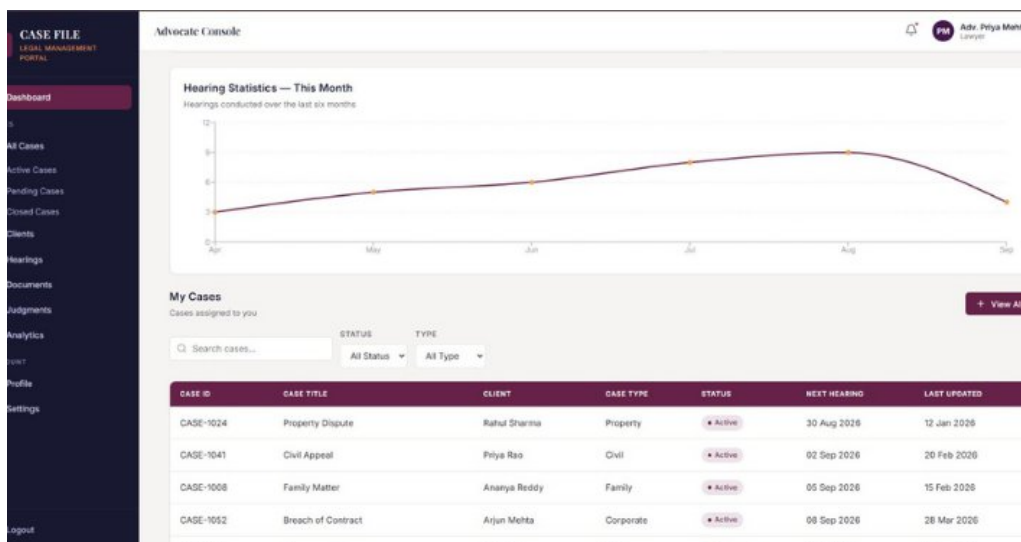
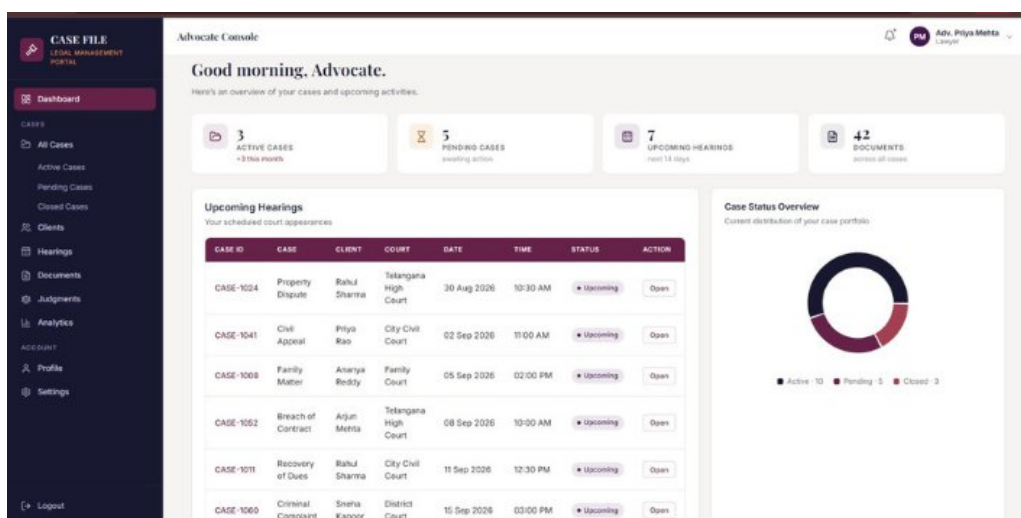
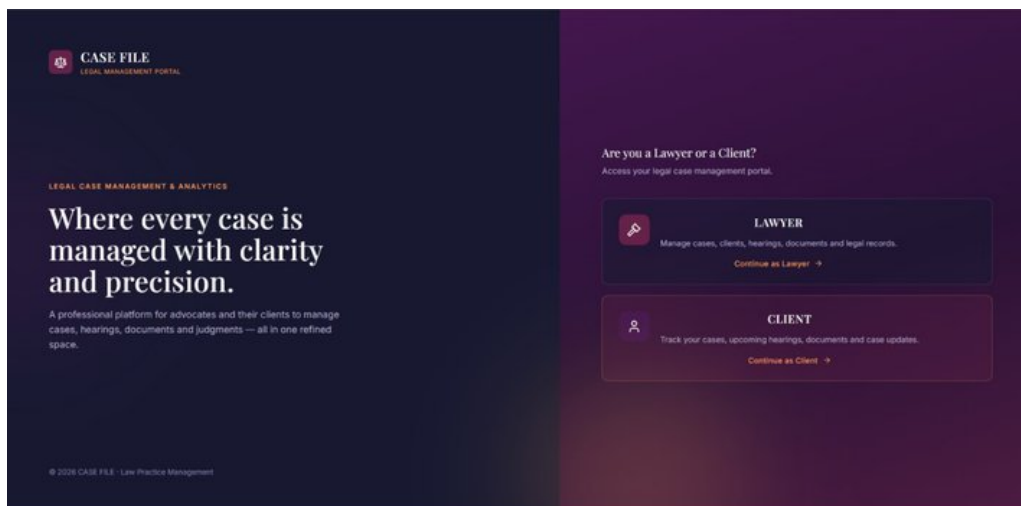
If code in a microservice changes, rebuild it with `..\..\apache-maven-3.9.6\bin\mvn.cmd clean package -DskipTests` (after stopping the old Java process so the JAR is not locked).

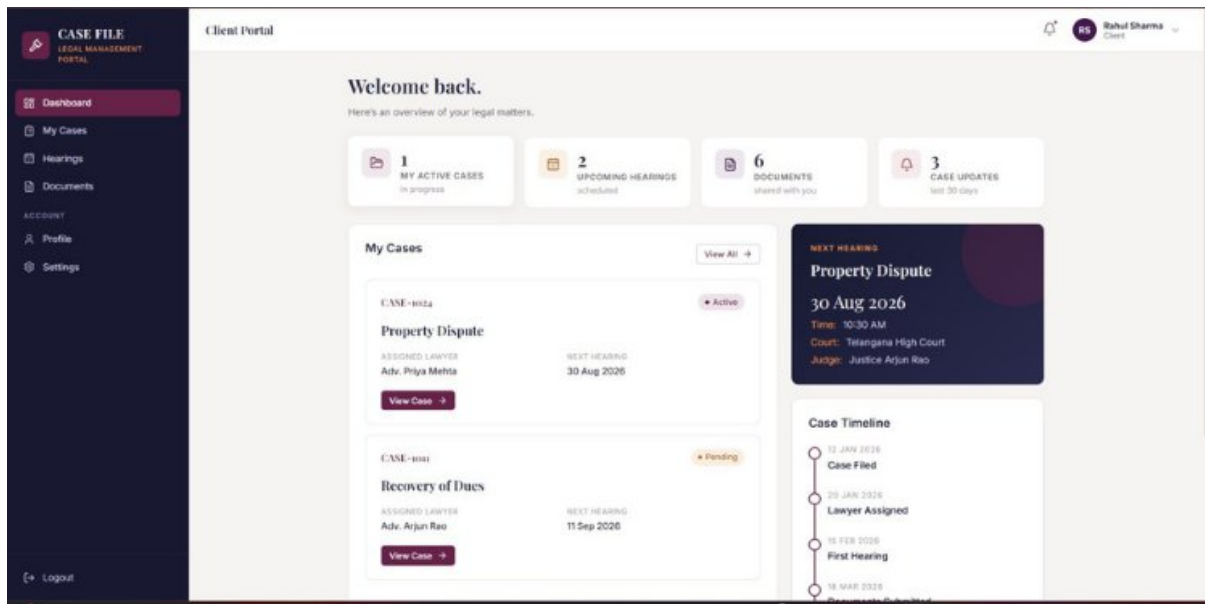
9.2 Environment Configuration

Each Spring Boot service keeps its own configuration (server port, MySQL connection details and the base URL of the service it calls) in its Spring configuration file, and the FastAPI backend keeps its database connection in its own configuration. The fixed ports are 8000 (FastAPI), 8081 (User), 8082 (Case), 8083 (Record) and 5173 (React). Real credentials should not be committed to version control.

9.3 Hosting of Frontend and Backend

For this submission, all components run as local processes and the database runs as a local MySQL instance, which is appropriate for development and in-class demonstration. A production deployment was not required; Chapter 11 outlines how the system could be extended to a hosted, containerised environment.





Project

```
589
590 • SELECT user_id, full_name, email, role
591 FROM users
592 WHERE email = 'ananya.test2026@example.com';
593
594 • SELECT * FROM cases;
```

Result Grid

case_id	client_id	lawyer_id	case_number	title	case_type	description	status	filing_date	court_name
35	1	16	CASE-10024-16	Property Dispute	Civil	Dispute regarding property ownership.	Active	2026-09-11	Telangana
36	1	16	CASE-10034-16	Cyber Fraud Complaint	Cyber Crime	Complaint regarding an online financial fraud inc...	Active	2026-09-11	Telangana
37	1	16	10000014-16	Inheritance Issue	Inheritance	Property inheritance dispute	high	2026-09-22	Supreme
38	1	16	CASE-10014-16	Inheritance Issue	Inheritance	Property inheritance dispute	high	2026-09-22	Supreme

```
532
533 • SELECT * FROM users;
534
535 • SELECT * FROM clients;
536
537 • SELECT * FROM lawyers;
538
```

Result Grid

judgment_id	case_id	judgment_date	decision	summary	remarks
1	2	2026-09-05	Case Pending	The court has reviewed the initial submissions.	Further hearing is required.
2	5	2026-09-06	Guilty	The court found sufficient evidence supporting ...	Appropriate legal action recommended.
3	10	2026-09-07	Pending Decision	The property ownership documents were revie...	Final decision will be announced later.
4	19	2026-09-10	Under Review	The cyber fraud complaint is currently under jud...	Additional evidence may be submitted.
NULL	NULL	NULL	NULL	NULL	NULL

users 5 clients 6 lawyers 7 judges 8 cases 9 hearings 10 documents 11 judgments 12 x

Result Grid

hearing_id	case_id	judge_id	hearing_date	hearing_time	court_room	purpose	status	remarks
6	2	1	2026-09-20	10:30:00	Court Hall 1	Initial Hearing	Completed	Initial arguments will be presented.
7	5	2	2026-09-22	11:00:00	Court Hall 2	Evidence Submission	Completed	Evidence documents will be submitted.
8	10	3	2026-09-24	12:00:00	Court Hall 3	Case Review	Postponed	Both parties will appear before the court.
9	19	4	2026-09-26	10:00:00	Court Hall 4	Investigation Review	Postponed	Investigation progress will be reviewed.
10	2	5	2026-10-07	11:30:00	Court Hall 5	Final Arguments	Upcoming	Final arguments will be presented.

users 5 clients 6 lawyers 7 judges 8 cases 9 hearings 10 x documents 11 judgments 12

Result Grid							
		Filter Rows:		Edit:		Export/Import:	
	document_id	case_id	document_name	document_type	uploaded_by	upload_date	file_path
▶	1	2	Case Application.pdf	Application	1	2026-09-11 09:30:00	uploads/case_application.pdf
	2	5	Police Report.pdf	Police Report	2	2026-09-11 09:30:00	uploads/police_report.pdf
	3	10	Property Deed.pdf	Legal Document	5	2026-09-11 09:30:00	uploads/property_deed.pdf
	4	19	Cyber Crime Complaint.pdf	Complaint	6	2026-09-11 09:30:00	uploads/cyber_complaint.pdf
	5	5	Evidence Document.pdf	Evidence	1	2026-09-11 09:30:00	uploads/evidence_document.pdf
users 5 clients 6 lawyers 7 judges 8 cases 9 hearings 10 documents 11 × judgments 12							

Result Grid						
		Filter Rows:		Edit:		Export/Import:
	judgment_id	case_id	judgment_date	decision	summary	remarks
▶	1	2	2026-09-05	Case Pending	The court has reviewed the initial submissions.	Further hearing is required.
	2	5	2026-09-06	Guilty	The court found sufficient evidence supporting ...	Appropriate legal action recommended.
	3	10	2026-09-07	Pending Decision	The property ownership documents were revie...	Final decision will be announced later.
	4	19	2026-09-10	Under Review	The cyber fraud complaint is currently under jud...	Additional evidence may be submitted.
*	NULL	NULL	NULL	NULL	NULL	NULL
users 5 clients 6 lawyers 7 judges 8 cases 9 hearings 10 documents 11 judgments 12 ×						

Result Grid				
		Filter Rows:		Edit:
	judge_id	full_name	court_name	appointment_date
▶	1	Justice Anil Kumar	Telangana High Court	2020-01-15
	2	Justice Priya Sharma	Telangana High Court	2019-06-20
	3	Justice Ramesh Rao	Hyderabad District Court	2021-03-15
	4	Justice Kavita Reddy	Secunderabad District Court	2018-11-10
	5	Justice Suresh Kumar	Cyberabad Court	2022-01-25
users 5 clients 6 lawyers 7 judges 8 × cases 9 hearings 10 documents 11 judgments 12				
Output :				

CHAPTER 10: CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

- **Analytics page showing zero cases.** The page appeared blank with 0 cases even though cases existed, because some of the microservices had been closed and could not answer requests.
- **Field-name mismatch.** The Spring Boot services return camelCase fields (caseId, caseNumber) while parts of the UI expected snake_case (case_id, case_number), so some values were missing.
- **Duplicate health endpoints.** Two controllers mapped the same health URL, which caused an “ambiguous mapping” error at startup.
- **Port conflicts and locked JAR files.** Starting a second copy of a service failed because the port was already in use, and rebuilding failed while an old Java process still held the JAR.
- **Seed-data inconsistencies.** While inserting test data, duplicate case numbers and foreign-key IDs that did not exist (for example a lawyer profile pointing at the wrong user_id) were rejected by the database.
- **“Failed to fetch” errors and lost sessions.** These appeared when a service had not been started or when FastAPI had been restarted, because sessions are stored in memory.

10.2 Solutions Applied

The analytics problem was solved by checking each service’s health endpoint, restarting the missing services in the documented order, and making the Analytics page show the actual error instead of silent zeros. The field-name mismatch was fixed by making formatCase (and the lawyer-ID lookup) accept both naming styles using nullish coalescing. The duplicate health mapping was fixed by removing the extra method; port and JAR-lock problems were fixed by identifying the existing process with netstat before starting another copy and stopping the old Java process before rebuilding. Seed-data errors were corrected by first querying the real client, lawyer, case and judge IDs and then inserting rows that referenced them. Finally, a HOW_TO_RUN_PROJECT.txt guide with start order, health checks and a troubleshooting decision tree was written so that the whole system can be restarted from scratch.

10.3 Current Limitations or Known Bugs

- Sessions are held in FastAPI’s memory, so restarting FastAPI requires users to log in again.
- Documents are stored as metadata and a file path; there is no physical file upload/download service.
- The three microservices must be started in order and kept running; there is no automatic service discovery or one-click launcher.

- No e-mail verification, password reset or notifications exist.
- The system has been tested on a single local deployment and has not been deployed to a public cloud environment.

CHAPTER 11: FUTURE ENHANCEMENTS

11.1 Planned Features

- A real document upload and download service, with secure file storage and access control per case.
- Hearing reminders and status-change notifications by e-mail or SMS for lawyers and clients.
- A calendar view of hearings so that a lawyer can see a day, week or month at a glance.
- Password reset, e-mail verification and an admin role for managing users.
- Billing and invoicing for legal services.

11.2 Possible Integrations or Optimizations

- Adding an API gateway so React talks to one entry point instead of four separate ports.
- Persisting sessions (for example in a database or Redis) so users stay logged in across FastAPI restarts.
- Containerising each service with Docker and orchestrating with Docker Compose for one-command startup.
- Adding automated tests (JUnit and Mockito for Spring Boot, pytest for FastAPI) and a CI pipeline.
- Giving each service its own database schema and adding caching for frequently requested analytics.

CHAPTER 12: CONCLUSION

12.1 Summary of the Project

This project set out to replace scattered, informal case tracking with a single, database-driven web application. Over the course of development, a normalised eight-table MySQL schema, a FastAPI authentication and analytics backend, three Spring Boot microservices communicating over REST, and a React frontend for lawyers and clients were designed, built and tested incrementally against a real, running database.

12.2 What Was Achieved

The finished system supports role-based login with hashed passwords; full management of cases, hearings, documents and judgments; service-level validation of relationships (a case needs a valid client and lawyer, and hearings, documents and judgments need a valid case) through REST calls between services; aggregated endpoints that combine data from two services; and an analytics dashboard that always reflects the current database after cases are added, edited or deleted. Existing data was preserved throughout testing.

12.3 Skills Learned

Working on this project provided practical experience in relational database design with keys and constraints; decomposing a system into microservices with clear data ownership; designing REST APIs and service-to-service communication with Spring Boot's RestClient; building a FastAPI backend with session-based authentication and password hashing; building a React frontend with Context-based state and charts; and diagnosing real integration problems — port conflicts, field-name mismatches, missing services and stale state — through systematic health checks rather than guesswork.

CHAPTER 13: REFERENCES

- React — Official Documentation, <https://react.dev>
- Vite — Official Documentation, <https://vitejs.dev>
- Recharts — Documentation, <https://recharts.org>
- FastAPI — Official Documentation, <https://fastapi.tiangolo.com>
- Uvicorn — Documentation, <https://www.uvicorn.org>
- SQLAlchemy — Documentation, <https://docs.sqlalchemy.org>
- Spring Boot — Reference Documentation, <https://docs.spring.io/spring-boot>
- Apache Maven — Documentation, <https://maven.apache.org>
- MySQL 8.0 Reference Manual, <https://dev.mysql.com/doc/refman/8.0/en/>
- Argon2 password hashing — <https://github.com/P-H-C/phc-winner-argon2>
- Postman Learning Center, <https://learning.postman.com>
- MDN Web Docs — JavaScript and Web APIs reference, <https://developer.mozilla.org>

CHAPTER 14: APPENDICES

Appendix A: Sample Code Snippets

A representative SQL statement used in the project — it creates a case by looking up the client and lawyer IDs from their user e-mail addresses instead of hard-coding them:

```
INSERT INTO cases (client_id, lawyer_id, case_number, title,
                  case_type, description, status, filing_date, court_name)
VALUES (
  (SELECT c.client_id FROM clients c
   JOIN users u ON c.user_id = u.user_id WHERE u.email = 'rahul@test.com'),
  (SELECT l.lawyer_id FROM lawyers l
   JOIN users u ON l.user_id = u.user_id WHERE u.email = 'lawyer@test.com'),
  'CASE-1024', 'Property Dispute', 'Property',
  'Dispute regarding residential property ownership.',
  'Active', '2026-08-01', 'Telangana High Court');
```

An example of the kind of query behind the analytics “cases by status” chart (illustrative):

```
SELECT status, COUNT(*) AS total
FROM cases
GROUP BY status;
```

Appendix B: Installation and Setup Instructions

- Make sure MySQL Server is running and the database legal_case_management exists (create it once with Project.sql; never re-run it on restart).
- Backend: cd backend, then .\venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000
- Microservices: in three separate terminals run java -jar target\user-service-1.0.0.jar, case-service-1.0.0.jar and record-service-1.0.0.jar from their folders.
- Frontend: cd case-file, then npm run dev (run npm install only if node_modules is missing).
- Open http://localhost:5173 in a browser.

Appendix C: Default Login Credentials

Sample accounts were inserted into the database for demonstration. The seeded client accounts below share the password “password123”.

Role	Email	Password
Client	rahul@test.com	password123
Client	priya@test.com, arjun@test.com	password123
Lawyer	lawyer@test.com	[fill in the password you set]
Lawyer	sneha@test.com, vikram@test.com	[fill in the password you set]

Appendix D: API Reference Summary

Service	Port	Main endpoints
FastAPI	8000	GET / ; register / login / session endpoints ; GET /analytics/overview (needs X-Session-Token)

Service	Port	Main endpoints
User Service	8081	GET /users/health ; users, GET /clients, GET /clients/{id} ; GET /lawyers, GET /lawyers/{id}
Case Service	8082	GET /cases/health ; GET, POST /cases ; GET, PUT, DELETE /cases/{id} ; GET /cases/{id}/with-users
Record Service	8083	GET /records/health ; GET, POST /records/hearings, /records/documents, /records/judgments ; GET, PUT, DELETE on /{id} ; GET /{id}/with-case

Appendix E: Screenshots

Insert screenshots from the running application on the following pages before submission. Suggested screenshots, in order:

- Login page
- Lawyer dashboard
- Client dashboard
- Cases page (list and add/edit case form)
- Hearings page
- Documents page, including the client's View action
- Judgments page
- Analytics page (before and after adding a case, to show live updates)
- Health-check output of all four services in PowerShell
- MySQL Workbench showing the eight tables and sample rows